

Credit Card Fraud Detection

Machine Learning Project Report

Contents

1	Introduction	4
2	Python Code	4
3	Code Explanation	6
3.1	Importing Libraries	6
3.2	Getting Script Directory	6
3.3	Reading CSV Files	6
3.4	Train vs Test Dataset	7
4	Business Objective	7
5	Target Variable	7
6	Target Distribution Analysis	7
6.1	Important Observation	8
6.2	Why Imbalanced Data is Dangerous	8
7	Dataset Overview	8
7.1	Explanation	8
8	Feature Descriptions	9
8.1	Important Features	9
8.1.1	Transaction Amount (amt)	9
8.1.2	Location Features	9
8.1.3	Transaction Time	10
9	Data Types Summary	10
9.1	Explanation	10
9.2	Important Observation	11

10 Missing Values Check	11
10.1 Why Missing Values Matter	11
11 Machine Learning Concepts Used	11
12 Conclusion	12
13 Part 2: Advanced Data Exploration	12
13.1 Python Code for Advanced Exploration	13
14 Advanced Statistics	13
14.1 Understanding describe()	13
14.2 Key Findings	14
15 Skewness Analysis	14
15.1 Results	14
15.2 Interpretation	14
15.3 Important Observation	15
16 Kurtosis Analysis	15
16.1 Results	15
16.2 Interpretation	15
17 Correlation Matrix	16
17.1 Results	16
17.2 Interpretation	16
18 Correlation Heatmap	16
19 Outlier Detection using IQR	17
19.1 Results for amt	17
19.2 Results for city_pop	18
19.3 Interpretation	18
20 Class Imbalance Analysis	18
20.1 Interpretation	18
20.2 Why This Matters	19
21 Visualizations Analysis	19
21.1 Amount Distribution	19
21.2 Top Fraud Categories	20
21.3 Gender Fraud Distribution	20
21.4 Fraud Amount Boxplot	21

21.5 Fraud by Hour	21
21.6 Explanation	22
21.7 Important Finding	22
22 FutureWarning Explanation	22
23 Overall Exploration Summary	22
24 Connection Between Part 1 and Part 2	23
25 Next Steps	23

1 Introduction

This project focuses on detecting fraudulent credit card transactions using Machine Learning.

Financial fraud causes major losses for banks and customers every year. The purpose of this project is to build a system capable of automatically identifying suspicious transactions.

This problem is classified as:

Binary Classification Problem

Where:

- 0 = Legitimate Transaction
- 1 = Fraudulent Transaction

2 Python Code

```
1  """
2  =====
3  Part 1: Problem Definition - Credit Card Fraud Detection
4  =====
5  """
6
7  import pandas as pd
8  import os
9
10 # -----
11 # 1. Load the Dataset
12 # -----
13 SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
14
15 df_train = pd.read_csv(os.path.join(SCRIPT_DIR, "fraudTrain.csv"))
16 df_test = pd.read_csv(os.path.join(SCRIPT_DIR, "fraudTest.csv"))
17
18 print("=" * 60)
19 print("    PART 1: PROBLEM DEFINITION")
20 print("=" * 60)
21
22 print("""
23 +-----+
```

```

24 | BUSINESS OBJECTIVE |
25 +-----+
26 | |
27 | The goal of this project is to build a machine |
28 | learning model that can accurately detect |
29 | fraudulent credit card transactions. |
30 | |
31 +-----+
32 """
33
34 print("=" * 60)
35 print(" TARGET VARIABLE: is_fraud")
36 print("=" * 60)
37
38 print(f"""
39 Column Name : is_fraud
40 Data Type : Integer (Binary)
41 Values : 0 = Legitimate Transaction
42          1 = Fraudulent Transaction
43 """)
44
45 print("=" * 60)
46 print(" TARGET VARIABLE DISTRIBUTION (Train Set)")
47 print("=" * 60)
48
49 target_counts = df_train["is_fraud"].value_counts()
50 target_percent = df_train["is_fraud"].value_counts(normalize=True) *
51 100
52
53 print(f"""
54 Total Transactions : {len(df_train):,}
55
56 Not Fraud (0) : {target_counts[0]:,} ({target_percent[0]:.2f
57 }%)
58 Fraud (1) : {target_counts[1]:,} ({target_percent[1]:.2f
59 }%)
60 """)
61
62 print("=" * 60)
63 print(" DATASET OVERVIEW")
64 print("=" * 60)
65
66 print(f"""

```

```
64 Train Set Size : {df_train.shape[0]:,} rows × {df_train.shape
    [1]} columns
65 Test Set Size : {df_test.shape[0]:,} rows × {df_test.shape[1]}
    columns
66 """)
```

3 Code Explanation

3.1 Importing Libraries

```
1 import pandas as pd
2 import os
```

Explanation:

- **pandas:** Used for data analysis and handling CSV files.
- **os:** Used for handling file paths and directories.

3.2 Getting Script Directory

```
1 SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
```

Purpose:

This line obtains the directory where the current Python script is located.

Why Important?

Without this step, the CSV files may not load correctly if the program is executed from another directory.

3.3 Reading CSV Files

```
1 df_train = pd.read_csv(os.path.join(SCRIPT_DIR, "fraudTrain.csv"))
2 df_test = pd.read_csv(os.path.join(SCRIPT_DIR, "fraudTest.csv"))
```

Explanation:

- **read_csv()** loads CSV files into DataFrames.
- **df_train** contains training data.
- **df_test** contains testing data.

3.4 Train vs Test Dataset

Dataset	Purpose
Training Set	Used to train Machine Learning models
Testing Set	Used to evaluate model performance on unseen data

4 Business Objective

The main objective is to detect fraudulent transactions automatically.

Banks use fraud detection systems to:

- Protect customers
- Reduce financial losses
- Improve customer trust
- Detect suspicious activity in real time

5 Target Variable

The target column is:

`is_fraud`

Value	Meaning
0	Legitimate transaction
1	Fraudulent transaction

This is a binary classification task because there are only two possible outcomes.

6 Target Distribution Analysis

Output:

Total Transactions : 1,296,675

Not Fraud (0) : 1,289,169 (99.42%)

Fraud (1) : 7,506 (0.58%)

6.1 Important Observation

The dataset is highly imbalanced.

Meaning:

Fraudulent transactions represent only 0.58% of all transactions.

6.2 Why Imbalanced Data is Dangerous

Imagine a model that predicts:

"Every transaction is NOT fraud"

The model would still achieve approximately 99% accuracy.

However, it would fail to detect actual fraud.

Therefore:

Accuracy alone is not enough

Important metrics later include:

- Precision
- Recall
- F1-Score
- ROC-AUC

7 Dataset Overview

Dataset dimensions:

Train Set Size : 1,296,675 rows × 23 columns

Test Set Size : 555,719 rows × 23 columns

7.1 Explanation

- Rows represent transactions.
- Columns represent features or attributes.

The dataset contains nearly 1.9 million records.

This is considered a large real-world financial dataset.

8 Feature Descriptions

Feature	Description
trans_date_trans_time	Transaction date and time
cc_num	Credit card number (customer identifier)
merchant	Merchant name where transaction occurred
category	Transaction category (e.g., grocery, gas)
amt	Transaction amount in USD
first	Cardholder first name
last	Cardholder last name
gender	Cardholder gender (M/F)
street	Cardholder street address
city	Cardholder city
state	Cardholder state
zip	Cardholder zip code
lat	Customer latitude
long	Customer longitude
city_pop	Population of cardholder's city
job	Cardholder job/occupation
dob	Cardholder date of birth
trans_num	Unique transaction ID
unix_time	Transaction time in unix format
merch_lat	Merchant latitude
merch_long	Merchant longitude
is_fraud	Target variable (0=Not Fraud, 1=Fraud)

8.1 Important Features

8.1.1 Transaction Amount (amt)

Fraudulent transactions often involve unusual amounts.

8.1.2 Location Features

The following features are important:

- lat
- long
- merch_lat

- merch_long

These help calculate the distance between customer and merchant.
Large distances may indicate suspicious activity.

8.1.3 Transaction Time

Transaction timestamps can help identify:

- unusual transaction times
- high transaction frequency
- suspicious customer behavior

9 Data Types Summary

Numerical features:

```
['Unnamed: 0', 'cc_num', 'amt', 'zip', 'lat',  
'long', 'city_pop', 'unix_time',  
'merch_lat', 'merch_long', 'is_fraud']
```

Categorical features:

```
['trans_date_trans_time', 'merchant',  
'category', 'first', 'last',  
'gender', 'street', 'city',  
'state', 'job', 'dob', 'trans_num']
```

9.1 Explanation

Machine Learning models work mainly with numerical data.

Categorical features usually need encoding techniques such as:

- Label Encoding
- One-Hot Encoding

9.2 Important Observation

Columns such as:

- trans_date_trans_time
- dob

are currently stored as strings (object type).

They should later be converted to datetime format using:

```
1 pd.to_datetime()
```

10 Missing Values Check

```
1 missing = df_train.isnull().sum()
2 total_missing = missing.sum()
```

Purpose:

Checks whether the dataset contains missing values.

Output:

[OK] No missing values found in the dataset!

10.1 Why Missing Values Matter

Most Machine Learning algorithms cannot handle missing values directly.

Common preprocessing techniques include:

- Filling missing values
- Removing rows
- Statistical imputation

Fortunately, this dataset contains no missing values.

11 Machine Learning Concepts Used

Concept	Meaning
Binary Classification	Predicting one of two classes
Target Variable	The value the model predicts

Features	Input variables used for prediction
Imbalanced Dataset	One class dominates the dataset
Numerical Data	Number-based data
Categorical Data	Text/category-based data
Missing Values	Empty or null data
Train/Test Split	Separate datasets for training and evaluation

12 Conclusion

This section successfully:

- Loaded the datasets
- Defined the business problem
- Identified the target variable
- Analyzed class distribution
- Examined dataset structure
- Checked feature types
- Verified missing values

The next steps in the project will likely include:

- Data preprocessing
- Feature engineering
- Handling class imbalance
- Model training
- Evaluation

13 Part 2: Advanced Data Exploration

This section focuses on deeper exploratory data analysis (EDA).

The goal is to:

- Understand statistical properties of the dataset
- Detect skewness and outliers

- Analyze fraud patterns
- Discover useful features for Machine Learning
- Visualize important relationships

13.1 Python Code for Advanced Exploration

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import os
6
7 SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
8 PLOTS_DIR = os.path.join(SCRIPT_DIR, "plots")
9 os.makedirs(PLOTS_DIR, exist_ok=True)
10
11 sns.set_style("whitegrid")
12 plt.rcParams["figure.figsize"] = (10, 6)
13 plt.rcParams["figure.dpi"] = 100
14
15 df = pd.read_csv(os.path.join(SCRIPT_DIR, "fraudTrain.csv"))
16
17 numerical_cols = ["amt", "lat", "long", "city_pop", "merch_lat", "
18     merch_long"]
19
20 print(df[numerical_cols].describe())
21 print(df[numerical_cols].skew())
22 print(df[numerical_cols].kurtosis())

```

14 Advanced Statistics

Output summary:

Mean transaction amount = 70.35

Maximum amount = 28948.90

Minimum amount = 1.00

14.1 Understanding describe()

The function:

```
1 df.describe()
```

returns important statistics such as:

Statistic	Meaning
count	Number of records
mean	Average value
std	Standard deviation (spread)
min	Minimum value
25%	First quartile
50%	Median
75%	Third quartile
max	Maximum value

14.2 Key Findings

- Average transaction amount is 70.35 dollars.
- Maximum transaction amount is extremely high.
- City population has very large variation.

This suggests the existence of:

- Outliers
- Skewed distributions

15 Skewness Analysis

Skewness measures asymmetry in data distribution.

15.1 Results

```
amt      : 42.2779 (Right-skewed)
city_pop : 5.5939  (Right-skewed)
lat      : -0.186  (Approximately Normal)
```

15.2 Interpretation

- Positive skewness means long tail to the right.
- Negative skewness means long tail to the left.
- Values near 0 indicate approximately normal distribution.

15.3 Important Observation

The feature:

`amt`

has extremely high skewness.

This means:

- Most transactions are small.
- Few transactions are extremely large.

This is common in financial datasets.

Later preprocessing may require:

```
1 np.log1p()
```

for log transformation.

16 Kurtosis Analysis

Kurtosis measures how heavy the tails of a distribution are.

16.1 Results

```
amt      : 4545.645 (Heavy-tailed)
city_pop : 37.6145  (Heavy-tailed)
```

16.2 Interpretation

Heavy-tailed distributions indicate:

- Extreme values
- High probability of outliers
- Unusual observations

This confirms that:

- transaction amounts contain extreme values
- city population contains major outliers

17 Correlation Matrix

Correlation measures the relationship between variables.

Values range between:

- $+1$ = strong positive relationship
- 0 = no relationship
- -1 = strong negative relationship

17.1 Results

```
amt          : 0.2194 (Moderate)
city_pop     : 0.0021 (Weak)
lat          : 0.0019 (Weak)
```

17.2 Interpretation

The feature:

amt

has the strongest correlation with fraud.

This suggests:

Higher transaction amounts are more likely to be fraudulent.

Other numerical features show weak direct correlation.

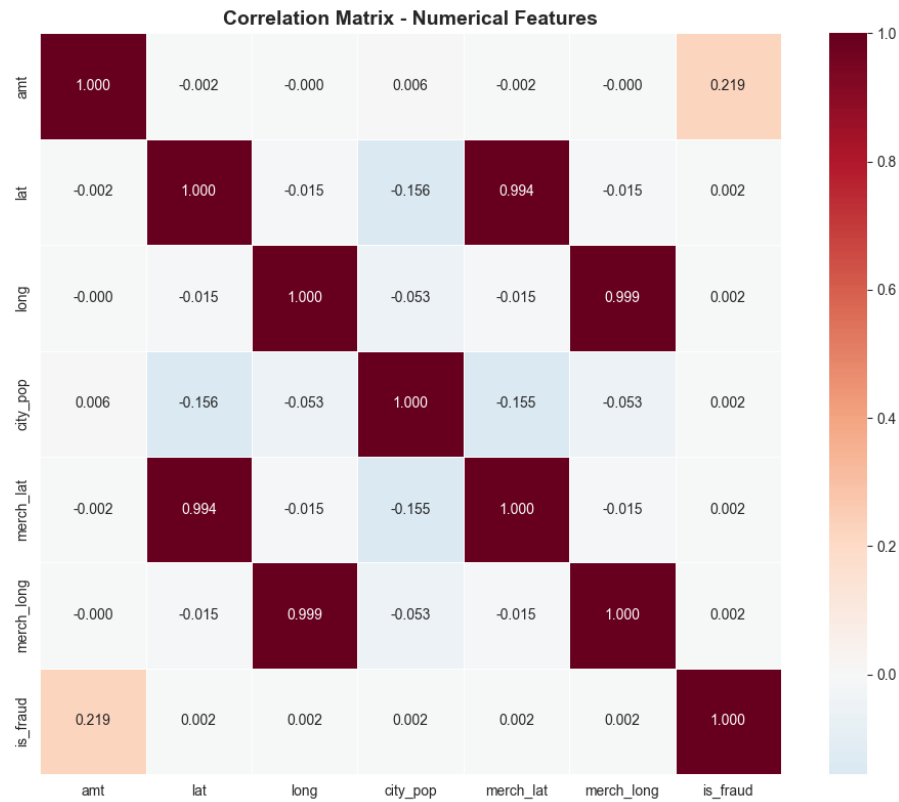
This does not mean they are useless.

Machine Learning models can still learn:

- nonlinear patterns
- interactions between variables

18 Correlation Heatmap

The following heatmap was generated:



Purpose:

- Visualize correlations between features
- Detect strongly related variables
- Understand feature interactions

19 Outlier Detection using IQR

Outliers were detected using the Interquartile Range (IQR) method.

Formula:

$$IQR = Q3 - Q1$$

$$LowerBound = Q1 - 1.5 * IQR$$

$$UpperBound = Q3 + 1.5 * IQR$$

19.1 Results for amt

Q1 = 9.65

Q3 = 83.14
IQR = 73.49
Outliers = 67,290 (5.19%)

19.2 Results for city_pop

Outliers = 242,674 (18.72%)

19.3 Interpretation

Outliers are very common in financial data.

Examples:

- unusually expensive purchases
- large city populations
- suspicious financial activity

Possible preprocessing techniques:

- log transformation
- clipping
- robust scaling

20 Class Imbalance Analysis

Results:

Not Fraud : 99.42%
Fraud : 0.58%
Imbalance Ratio = 1:171

20.1 Interpretation

For every:

1 fraudulent transaction

there are approximately:

171 normal transactions

This is an extremely imbalanced dataset.

20.2 Why This Matters

Without handling imbalance:

- the model may ignore fraud cases
- accuracy becomes misleading
- recall may become poor

Techniques that may be used later:

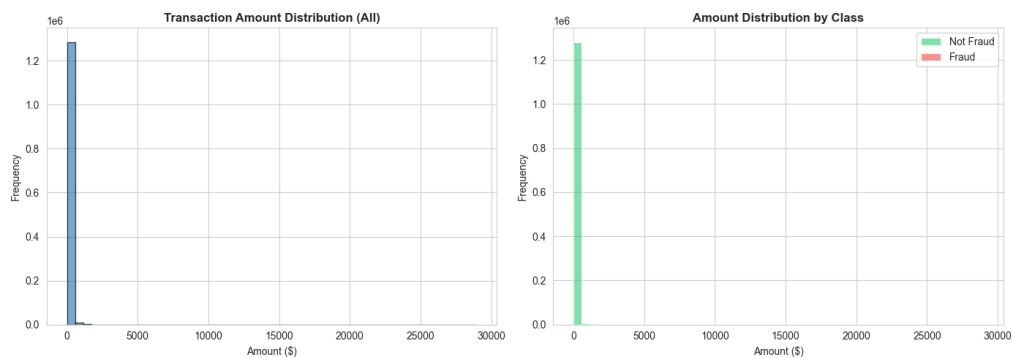
- SMOTE
- Random Oversampling
- Undersampling
- Class Weights

21 Visualizations Analysis

Several plots were generated during exploration.

21.1 Amount Distribution

File:

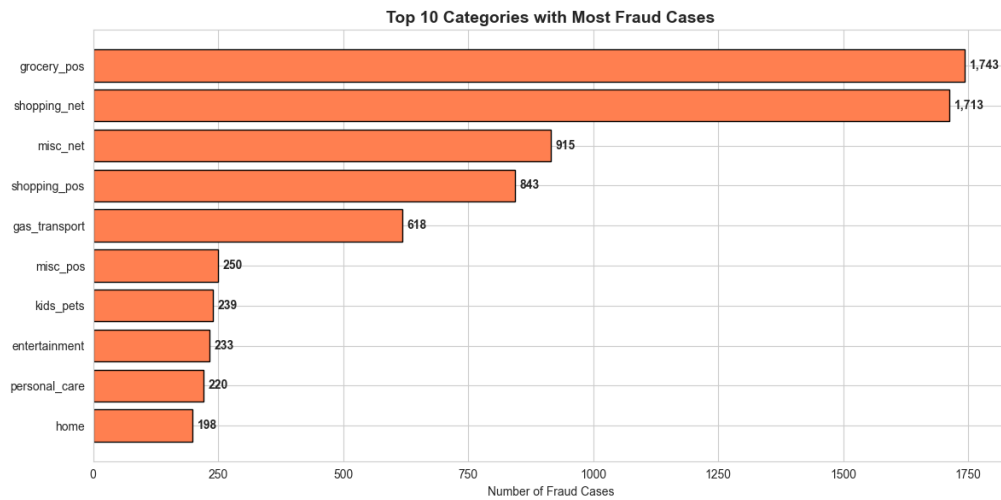


Observation:

- Most transactions are small.
- Fraud transactions tend to involve higher amounts.

21.2 Top Fraud Categories

File:



Purpose:

Identify transaction categories with highest fraud frequency.

This helps discover risky merchant categories.

21.3 Gender Fraud Distribution

File:



Purpose:

Analyze fraud distribution across genders.

21.4 Fraud Amount Boxplot

File:



Purpose:

Compare transaction amounts between:

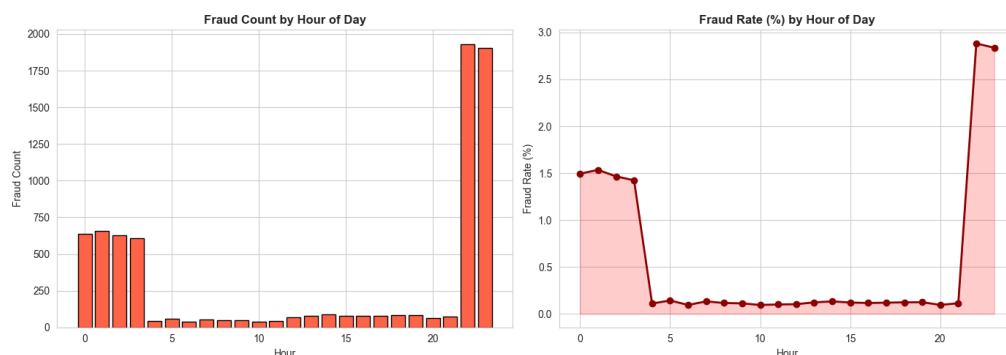
- Fraudulent transactions
- Legitimate transactions

The boxplot confirms:

Fraud transactions generally involve larger amounts.

21.5 Fraud by Hour

File:



The following code extracted transaction hour:

```
1 df["trans_hour"] = pd.to_datetime(df["trans_date_trans_time"]).dt.  
   hour
```

21.6 Explanation

- Convert transaction timestamp into datetime
- Extract hour only
- Analyze fraud behavior over time

21.7 Important Finding

Fraud rate changes throughout the day.

This means:

Time is an important predictive feature.

22 FutureWarning Explanation

The following warning appeared:

FutureWarning: Passing palette without assigning hue is deprecated

This warning comes from Seaborn.

It means future versions will require specifying:

```
1 hue=
```

inside the boxplot.

The current code still works correctly.

23 Overall Exploration Summary

Main findings from Part 2:

1. Dataset is highly imbalanced.
2. Fraud transactions tend to have higher amounts.
3. Amount and city population are highly skewed.
4. Significant outliers exist.
5. Fraud behavior changes by time.
6. Transaction category is important.

24 Connection Between Part 1 and Part 2

Part 1 focused on:

- Understanding the business problem
- Loading the dataset
- Identifying the target variable
- Checking missing values

Part 2 expanded the analysis by:

- Exploring statistical properties
- Detecting outliers
- Measuring skewness and kurtosis
- Visualizing fraud patterns
- Discovering important predictive features

Together, Part 1 and Part 2 prepare the project for:

- preprocessing
- feature engineering
- machine learning modeling

25 Next Steps

The next stage of the project will likely include:

- Handling class imbalance
- Feature engineering
- Encoding categorical variables
- Scaling numerical features
- Model training
- Model evaluation